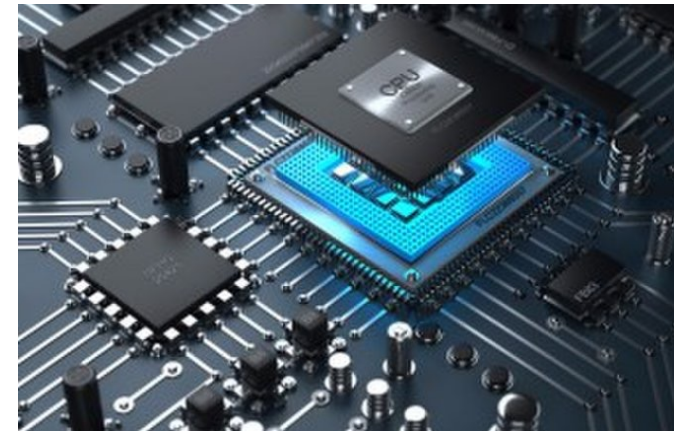


```
loop: lw    $t3, 0($t0)
      lw    $t4, 4($t0)
      add   $t2, $t3, $t4
      sw    $t2, 8($t0)
      addi  $t0, $t0, 4
      addi  $t1, $t1, -1
      bgtz  $t1, loop
```

Assembler

```
0x8d0b0000
0x8d0c0004
0x016c5020
0xad0a0008
0x21080004
0x2129ffff
0x1d20fff9
```



Lecture 09:

CPU Memory Addressing

MIPS Instruction Assembly

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Administrative

Current homework due TODAY!

New homework will be out on Canvas soon!

New readings for this week!

- **Handout 4** on Instruction Assembly

Worksheet for this lecture available on Canvas

Reminder: Office Hours R 4 U!!!!

Rules of Memory Addressing

RULES:

In CPUs (including MIPS), memory is addressed in **BYTES**

Recall : In MIPS, memory addresses are made up of 32 bits each

Recall: an integer is 32 bits in size, i.e., its size is 4 Bytes

So, if 2 integers that are placed *next to each other (i.e., contiguous)* in memory
and the 1st one (**int X**) has address **0x1000 1000**, therefore
the **int Y** has address... **0x1000 1004**



What's at address **0x1000 1001**?

- ANSWER: That's **not** a valid address (in this example)!!! 😊

Other notes:

- Memory addresses are always **positive numbers**
- 32-bit addresses, means there's **2^{32} different addresses**
- But EACH address is for a BYTE in memory
- So, there are **2^{32} BYTES of memory** in MIPS!

Memory Range in MIPS

1) Do the addresses map onto *bit* values in memory? Or *word* values? Or...???

They map onto *BYTE values*! 🤔

Each address points to a BYTE of data!

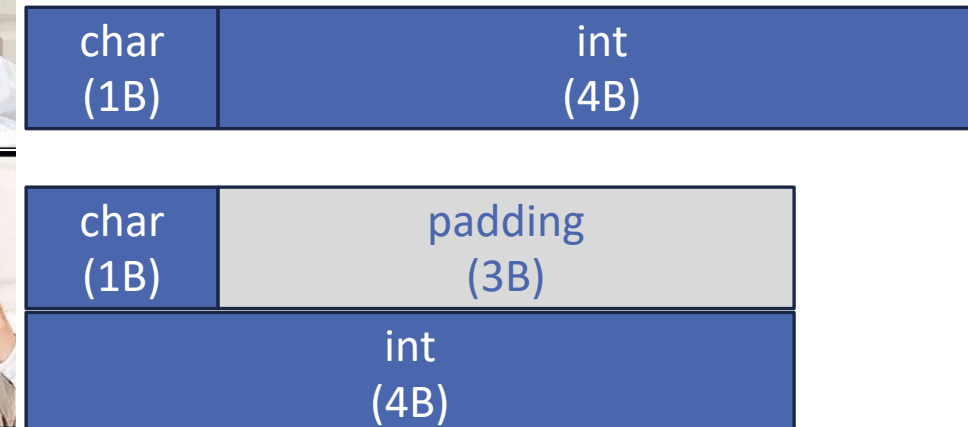
2) Are ALL 2^{32} addresses useable?

No! And for various reasons... Here's two:

- Some of CPU memory is reserved for use by the Operating System only
- For architectural & HW design reasons (and for running CPUs more *efficiently*), we have rules about WORD ALIGNMENT



WORD ALIGNMENT



Memory Range in MIPS

Since each address points to a **BYTE** of data...

... How ***much*** data is in MIPS CPU memory?

2^{32} Addresses each for 1 Byte...

...so, 2^{32} Bytes of memory!

2^{32} Bytes = $(2^2 \times 2^{30})$ Bytes = **4 Gibi-Bytes*** of CPU Memory in MIPS

$2^{10} \equiv$ kibi-

$2^{20} \equiv$ Mebi-

$2^{30} \equiv$ Gibi-

***NOTE:**

While $10^3 = 1$ kilo (i.e., 1,000)

$2^{10} = 1024 = 1$ kibi

Examples:

2^{10} bits = 1 kibi-bit

4×2^{20} bytes = 4 Mebi-Bytes

8×2^{30} bits = 8 Gibi-bits (*how many Gibi-Bytes is that??*)

What the !@\$##* is a Gibi-Byte????!!!



Memory Allocation Map

MEMORY ALLOCATION

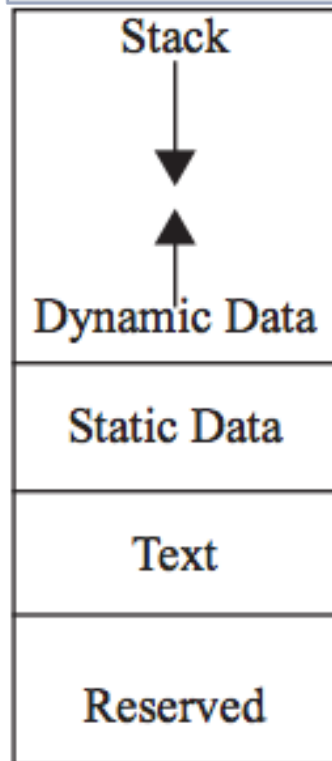
\$sp → 7fff fffc_{hex}

\$gp → 1000 8000_{hex}

1000 0000_{hex}

pc → 0040 0000_{hex}

0_{hex}



*This is found on your
MIPS Reference Card*

NOTES:

The end of the mem address shown here is not **0xF FFF FFFF** - what gives?

Yes, there's a total of **4 GiB** of memory, **but not all** memory addresses can be accessed *by the programmer*.

The top addresses from **0x8000 0000** up to **0xFFFF FFFF** are not available to user programs. *These are for use by the OS.*

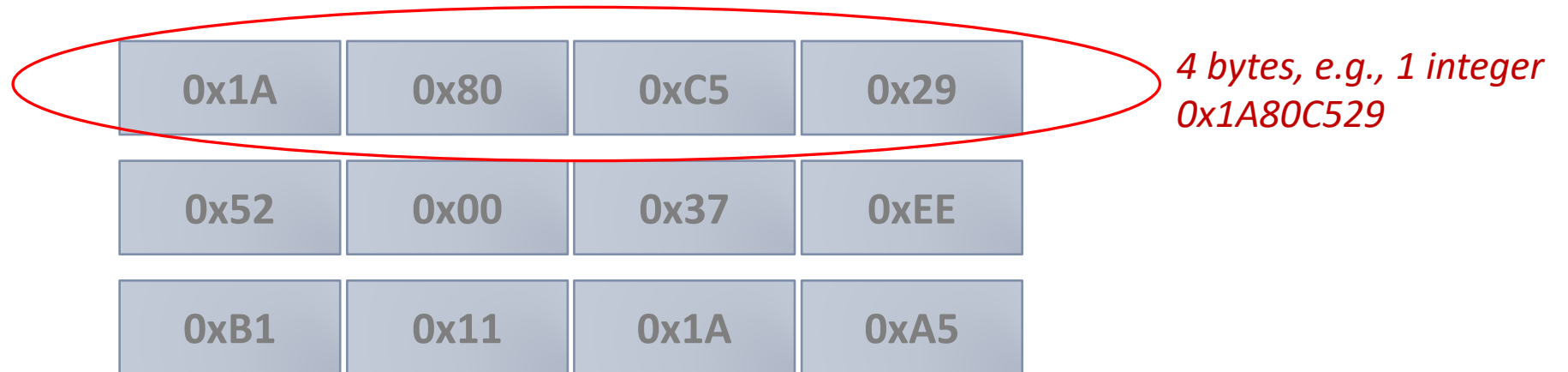
The same goes for the addresses from 0 to 0x0040 0000 (labeled "Reserved")

Byte-sized memory!



Each of these contiguous bytes has a contiguous address

These are just *example values* inside each **byte** of addressable memory.
Shown as hexadecimal values (1 byte has 8 bits $\equiv$ 2 hex digits)

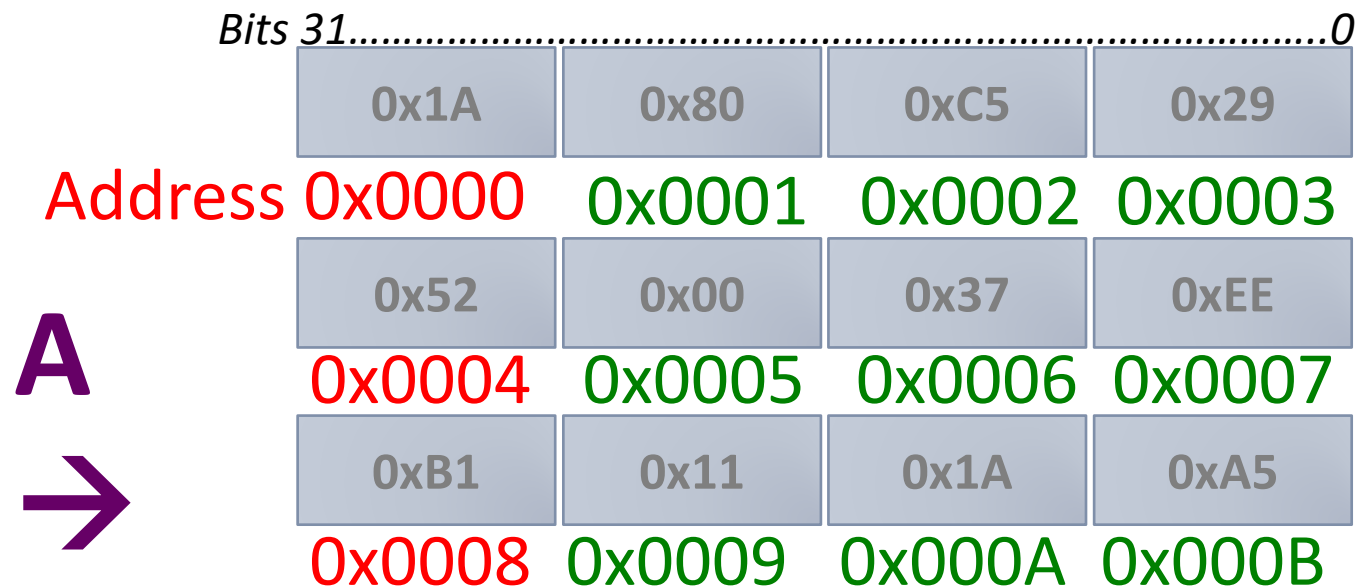


*It is common to organize them in groups of 4 bytes, i.e. in groups of **words***

Not everything we store in memory is a word long (but many things are)

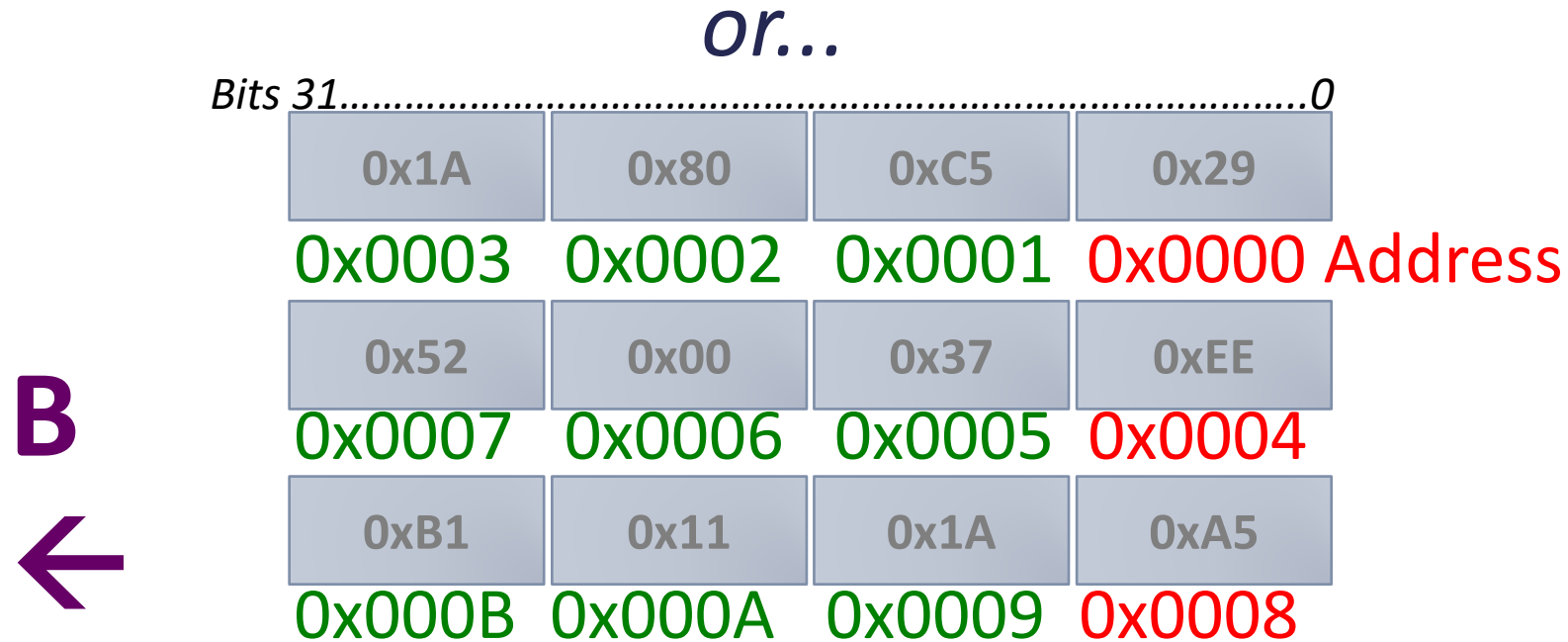
*In any case, this sets a **word alignment restriction** in how we store some data structures*

MIPS Computer Memory Addressing Conventions



Convention A: Addressing starts at the *most-significant byte* in each **word**

MIPS Computer Memory Addressing Conventions



Convention B: Addressing starts at the *least-significant **byte*** in each **word**

A Tale of 2 Conventions...

**BIG END (MSByte)
gets addressed first**

1A	80	C5	29
0x0000	0x0001	0x0002	0x0003
52	00	37	EE
0x0004	0x0005	0x0006	0x0007
B1	11	1A	A5
0x0008	0x0009	0x000A	0x000B

← **BIG ENDIAN**

LITTLE ENDIAN →

**LITTLE END (LSByte)
gets addressed first**

1A	80	C5	29
0x0003	0x0002	0x0001	0x0000
52	00	37	EE
0x0007	0x0006	0x0005	0x0004
B1	11	1A	A5
0x000B	0x000A	0x0009	0x0008

The Use of Big Endian vs. Little Endian

MIPS users typically go with **Big Endian** convention

- MIPS (and most CPUs) allows you to program “endian-ness”
- We won’t mess with that in CS 64... so we’ll stick w/ **Big Endian** convention in this course

Most Intel users (x86 CPUs) go with Little Endian...

It’s just a convention – it makes no difference to a CPU!

- The processes, rules, etc... remain the same

Addressing Memory – Recap

MIPS addresses are sequential, but the “addressing” is not done per (i.e., references) **words** of data

- Instead, addresses are references to Bytes of data

MIPS **words** stored in memory *must* start at addresses that are **multiples of 4** (why?)

- Because of the ***word alignment restriction***

We have to pick between 2 ***conventions*** of how to address words:
at the Most-Significant Byte? Or at the Least-Significant Byte?

- In this course, we will use the *former* convention (i.e. the MSB) – it’s called **Big Endian**

iClicker!

In the program shown here, you are told that the instruction `addi $t0, $zero, 1` is at memory address `0x0040 0120`

What is the address of instruction `add $v0, $t1, $t0` ?

- A. `0x0040 0124`
- B. `0x0040 012A`
- C. `0x0040 012C`
- D. `0x0040 012F`
- ☒ E. `0x0040 0130`

```
...  
0x0040 0120 ..... addi $t0, $zero, 1  
0x0040 0124 ..... addi $t1, $zero, 9  
0x0040 0128 ..... add $a0, $t0, $t1  
0x0040 012C ..... sll $t1, $t0, 5  
0x0040 0130 ..... add $v0, $t1, $t0  
0x0040 0134 ..... addi $v0, $v0, -9  
...
```

Disassembly

aka Turning Assembly into Machine Language

EXAMPLE: `add $t0, $s1, $s2`
 (recall syntax: `add rd, rs, rt`)

BASIC INSTRUCTION FORMATS

R	opcode	rs	rt	rd	shamt	funct						
	31	26	25	21	20	16	15	11	10	6	5	0
I	opcode	rs	rt	immediate								
	31	26	25	21	20	16	15	0				
J	opcode	address										
	31	26	25	0								

REGISTER NAME, NUMBER

op 6b	rs 5b	rt 5b	rd 5b	shamt 5b	funct 6b	REGISTER NAME, NUMBER
000000	10001	10010	01000	00000	100000	
						\$v0-\$v1 2-3
						\$a0-\$a3 4-7
						\$t0-\$t7 8-15
						\$s0-\$s7 16-23
						\$t8-\$t9 24-25
						\$k0-\$k1 26-27
						\$gp 28
						\$sp 29
						\$fp 30
						\$ra 31

00000010001100100100000000100000

0000 0010 0011 0010 0100 0000 0010 0000

`add $t0, $s1, $s2` → 0x02324020

R	opcode	rs	rt	rd	shamt	funct
	31 26 25	21 20	16 15	11 10	6 5	0
I	opcode	rs	rt	immediate		
	31 26 25	21 20	16 15	0		
J	opcode	address				
	31 26 25	0				

R-Type Instruction Representation

Recall: A MIPS instruction has 32 bits

The ***R-Type*** format takes the 32 instruction bits & divides them up into **6 fields**:

op code	6 bits	basic operation
rs code	5 bits	first register source operand
rt code	5 bits	second register source operand
rd code	5 bits	register destination operand
shamt code	5 bits	shift amount
funct code	6 bits	function code

*Why did the designers allocate **5 bits** for registers?*

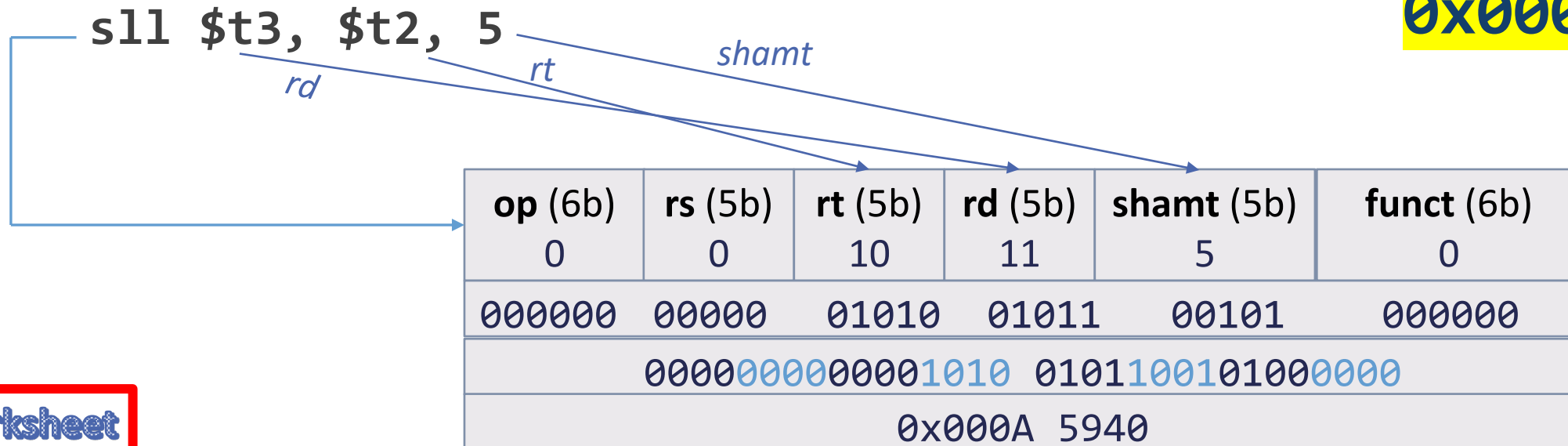
op	rs	rt	rd	shamt	funct
6 b	5 b	5 b	5 b	5 b	6 b
Bit number: 31 – 26	25 – 21	20 – 16	15 – 11	10 – 6	5 – 0

Note how the order of these DOES NOT MATCH the order of the assembly syntax!!

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	No

Exercise

Using your MIPS Reference Card, write the 32 bit *machine* instruction (using the **R-Type** format) for the following assembly instruction. Express your final answer in *hexadecimal*.



0x000A5940

R	opcode	rs	rt	rd	shamt	funct
	31 26 25	21 20	16 15	11 10	6 5	0
I	opcode	rs	rt	immediate		
	31 26 25	21 20	16 15	0		
J	opcode	address				
	31 26 25	0				

I-Type Instruction Representation

The ***I-Type*** format takes the 32 instruction bits & divides them up into **4 fields**:

op code	6 bits	basic operation
rs code	5 bits	first register source operand
rt code	5 bits	second register source operand
immediate code	16 bits	immediate value

op	rs	rt	immediate
6 b	5 b	5 b	16 b
Bit number: 31 – 26	25 – 21	20 – 16	15 – 0

Note how the order of these DOES NOT MATCH the order of the assembly syntax!!

I-Type Instruction Representation

The I-Type *immediate* field (16 bits) is a signed number

The **addi** instruction is an I-Type, example:

`addi $t0, $t1, 42`

What is the largest, most positive, number you can put as an immediate?

Ans: $2^{15} - 1 = 32,767$

NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	No

I-Type Instruction Representation

The **opcode** tells the processor what it is supposed to be doing (I-Types do not have **funct** or **shamt** codes to worry about)

EXAMPLE: **addi \$t0, \$s0, 129** (recall syntax: **addi rt, rs, immed**)

op = 8 mean "addi"

rs = 16 means "\$s0"

rt = 8 means "\$t0"

immed = 129

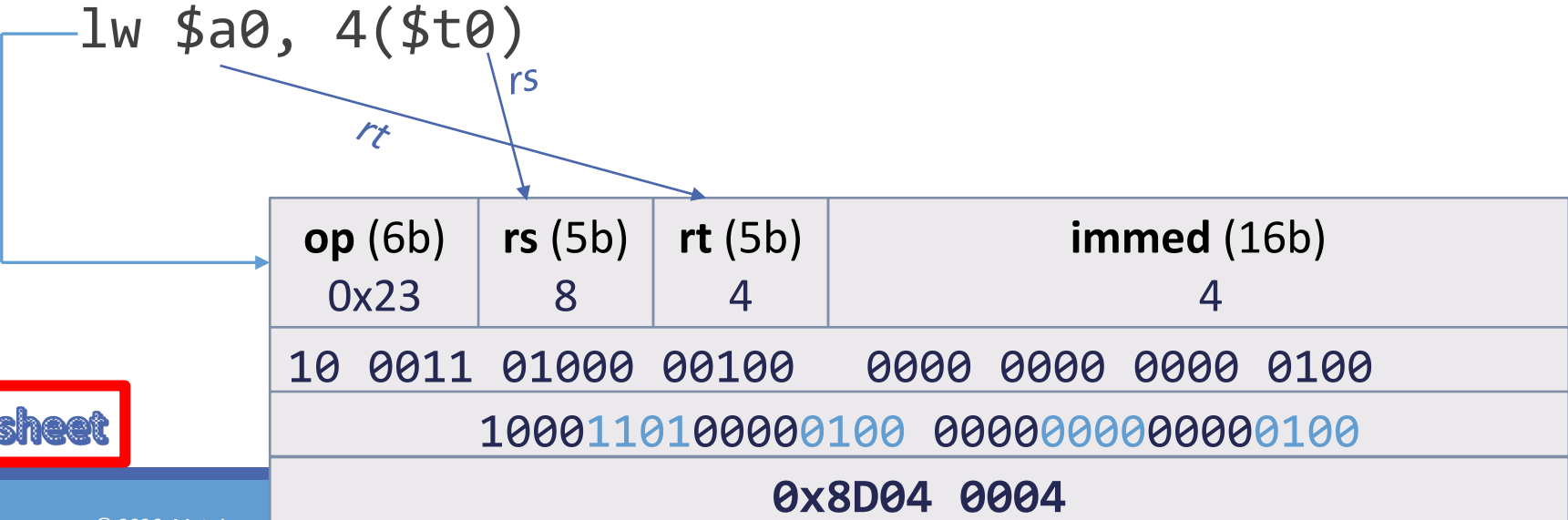
op	rs	rt	immediate
8	16	8	129
001000	10000	01000	0000 0000 1000 0001

REGISTER NAME, NUMBER, USE, CALL CONVENTION			
NAME	NUMBER	USE	PRESERVED ACROSS A CALL?
\$zero	0	The Constant Value 0	N.A.
\$at	1	Assembler Temporary	No
\$v0-\$v1	2-3	Values for Function Results and Expression Evaluation	No
\$a0-\$a3	4-7	Arguments	No
\$t0-\$t7	8-15	Temporaries	No
\$s0-\$s7	16-23	Saved Temporaries	Yes
\$t8-\$t9	24-25	Temporaries	No
\$k0-\$k1	26-27	Reserved for OS Kernel	No
\$gp	28	Global Pointer	Yes
\$sp	29	Stack Pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return Address	No

Exercise

Using your MIPS Reference Card, write the 32 bit *machine* instruction (using the **I-Type** format) for the following assembly instruction. Express your final answer in *hexadecimal*.

0x8D040004



Notes on OPICODE

Every MIPS instruction has a *unique* opcode

- Some **R**-types have a common **opcode** = 0, but then have unique **funct** values

Every MIPS instruction's opcode is in the **FIRST 6 BITS**

- Regardless of format type (i.e. whether R, I, or J)

Practice Exercises to Try at Home

Exercises for You to Try

1

Using your MIPS Reference Card, write the 32 bit instruction in binary and in hexadecimal for:

add \$t3, \$t2, \$s0

addu \$a0, \$a3, \$t0

sub \$t1, \$t1, \$t2

ANSWERS:

0x01505820

0x00E82021

0x012A4822

Exercises for You to Try

2

Using your MIPS Reference Card, write the 32 bit instruction in binary and in hexadecimal for:

`addi $t3, $t2, -42`

`slti $t8, $t8, 14`

`andi $a0, $a3, 1`

ANSWERS:

`0x214BFFD6`

`0x2B18000E`

`0x30E40001`

Assembling Exercise

1

Let's say you are given this MIPS machine language instruction: **0x01AE4821**

How do you find out the **assembly instruction** from it? (i.e. reverse process!)

1. Write it out in 32 bits: **0000 0001 1010 1110 0100 1000 0010 0001**
2. Realize that the 1st six (6) bits are the opcode (true of any MIPS instruction!!)

So, that means the **opcode** = 000000

Consult the MIPS Ref. Card → **opcode** = 0 means it's an R-type

So, you now need to figure out the **funct** field (*b/c multiple R-type instrs have opcode = 0*)

Assembling Exercise

2

0000 0001 1010 1110 0100 1000 0010 0001

2. Realize that the 1st six (6) bits are the opcode (in any MIPS instr.)

So, that means the **opcode** = 000000

Consult the MIPS Ref. Card → **opcode** = 0 means it's an R-type

And you now need to figure out the **funct** field

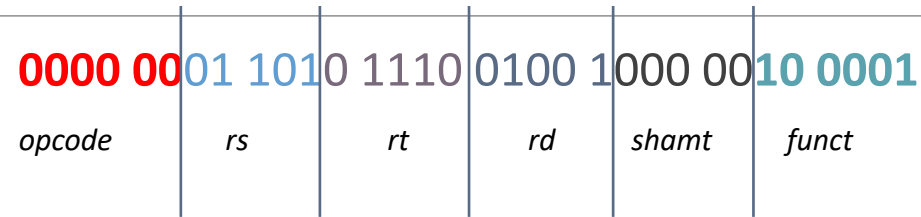
2b. The **funct** field is the last 6 bits in an R-type → **funct** = 10 0001 = 0x21 (better to write it in hex)

Again, consult the MIPS Ref. Card: what has **opcode** = 0 & **funct** = 0x21?

→ instruction is an addu

Assembling Exercise

3



3. Break up your binary instruction into its 6 R-type fields – WATCH OUT FOR THE CORRECT ORDER!!

opcode is first 6 bits, value = 0

rs is next 5 bits, value = 01101 = 13 → register **\$t5** (use the MIPS Ref.Card!)

rt is next 5 bits, value = 01110 = 14 → register **\$t6** (use the MIPS Ref.Card!)

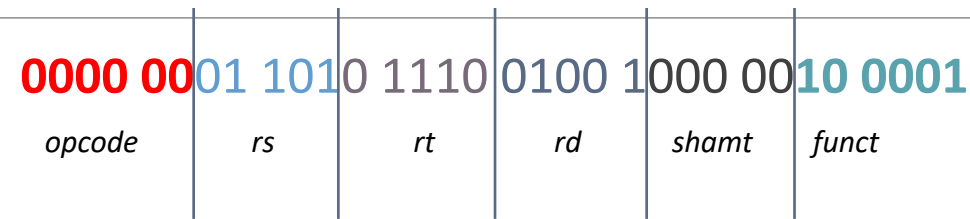
rd is next 5 bits, value = 01001 = 9 → register **\$t1** (use the MIPS Ref.Card!)

shamt is next 5 bits, value = 0 (predictably b/c there is no bit-shifting in addu)

funct is last 6 bits, value already found earlier

Assembling Exercise

4



4. Put 'em all together:

addu with $rs = \$t5$, $rt = \$t6$, $rd = \$t1$

Recall that the syntax for **addu** (and most R-Types) is: **addu** *rd*, *rs*, *rt*

Therefore, the instruction is: **addu \$t1, \$t5, \$t6**

To Dos

Turn in current assignment today

Look for new assignment

Read the handout

</LECTURE>